

TECHNICAL DEEP-DIVE • 20 MIN

Building an **AI Workspace**

Where Agents, MCP, Skills, and RAG come together to do real work — not just answer questions.





DEFINITION

What is an AI Workspace?

Not a chatbot. Not a single script. It's an environment where an agent connects to your tools, uses skills, retrieves your data, and completes real, multi-step tasks.



A MODEL IN A BOX

Just an LLM call

- Answers one question at a time
- Knows nothing about your data
- Can't touch your systems
- You copy-paste everything by hand



AN AI WORKSPACE

An agent that works

- Reasons through multi-step tasks
- Retrieves your docs/data (RAG)
- Connects to tools via MCP
- Acts, with guardrails + audit trail



THE MAP

Four pillars of an AI Workspace

Assemble these four and you have a workspace that does real work. We'll build each one — and demo two.



PILLAR 1

Agent

The brain — reasons, decides, orchestrates the loop.



PILLAR 2

MCP

The connectors — standard plug for tools & data.



PILLAR 3

Skills / Tools

The hands — capabilities the agent can invoke.



PILLAR 4

RAG

The knowledge — grounded in your own data.



STAND IT UP FIRST

Step 0 — the workspace environment

- A virtualenv + the agent stack + keys in .env. ~5 minutes.
- Pin dependencies so it's reproducible for the team.
- Add an observability key (LangSmith/Langfuse) on day one.
- Editor + a terminal you can demo from. That's the whole setup.

Boring but essential: a clean, shared environment is what turns “works on my laptop” into a team capability.

terminal

```
# 1. isolated environment
python -m venv .venv && source .venv/bin/activate

# 2. the workspace stack
pip install openai anthropic \
    langgraph fastmcp chromadb \
    instructor rank-bm25 ragas

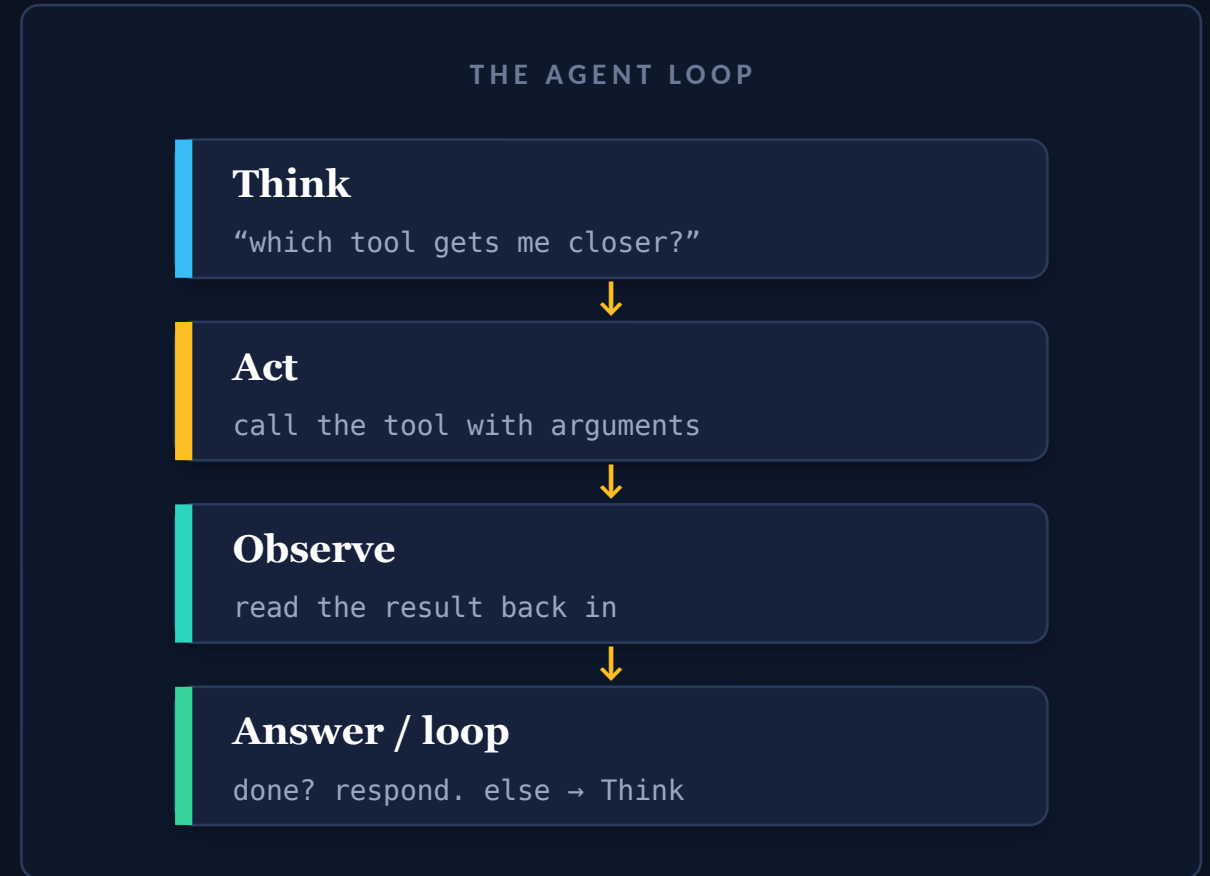
# 3. .env (keys)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
LANGCHAIN_TRACING_V2=true # traces
```



The Agent — a reasoning loop

- **The agent is the orchestrator:** it decides what to do next.
- ReAct loop: think → act (call a tool) → observe → repeat.
- Stops when it can answer — or when a guard trips.
- No framework needed: a prompt, a parser, a while-loop.

Production guards (non-negotiable): max-iterations • timeout • \$ budget • stuck-detection.





MCP — “USB-C for AI”

Model Context Protocol: one standard interface so any host — Claude Desktop, Claude Code, an IDE — can use your tools and data. Write a connector once; reuse it everywhere.



Tools

Functions the agent can call — query a DB, hit an API, run a job.



Resources

Data the agent can read — files, DB rows, API responses.



Prompts

Reusable prompt templates with arguments, shared across apps.

HOST (Claude Code / Desktop) → MCP CLIENT → YOUR MCP SERVER (tools · resources · prompts)



Build an MCP server in ~10 lines

my_mcp_server.py

```
# pip install fastmcp
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("my-tools-server")

@mcp.tool()
def query_db(sql: str, limit: int = 10) -> str:
    """Run a read-only SELECT, return JSON."""
    return run_select(sql, limit)

@mcp.resource("file://docs/{name}")
def get_doc(name): return read(name)

mcp.run() # stdio (local) or HTTP+SSE (remote)
```

CONNECT IT

Add to your host's config — that's it:

config.json

```
"mcpServers": {
  "my-tools": {
    "command": "python",
    "args": ["server.py"]
  }
}
```

Reuse win: the same tool now works in every app and model — write once.



Skills & Tools — give the agent capabilities

- A “skill” = a capability you grant the agent = a tool.
- Function + schema (name, description, params) → register it.
- The description IS the UX — it decides if the agent picks it right.
- Harden it: sandbox / allowlist / read-only · logging · rate-limit.

Tool description quality = agent reliability. Write tools like you write docs — for a smart but literal reader.

skills · registry

```
registry = ToolRegistry()

@registry.register({
    "name": "search_web",
    "description":
        "Search the web for
        current info.",
    "parameters": {...},
})
def search_web(query): ...

# give the agent its hands:
tools = registry.get_schemas()
```




RAG — ground it in your data

Give the workspace memory of YOUR world — docs, tickets, wikis. Retrieve the right chunks into context, answer with citations, cut hallucination.

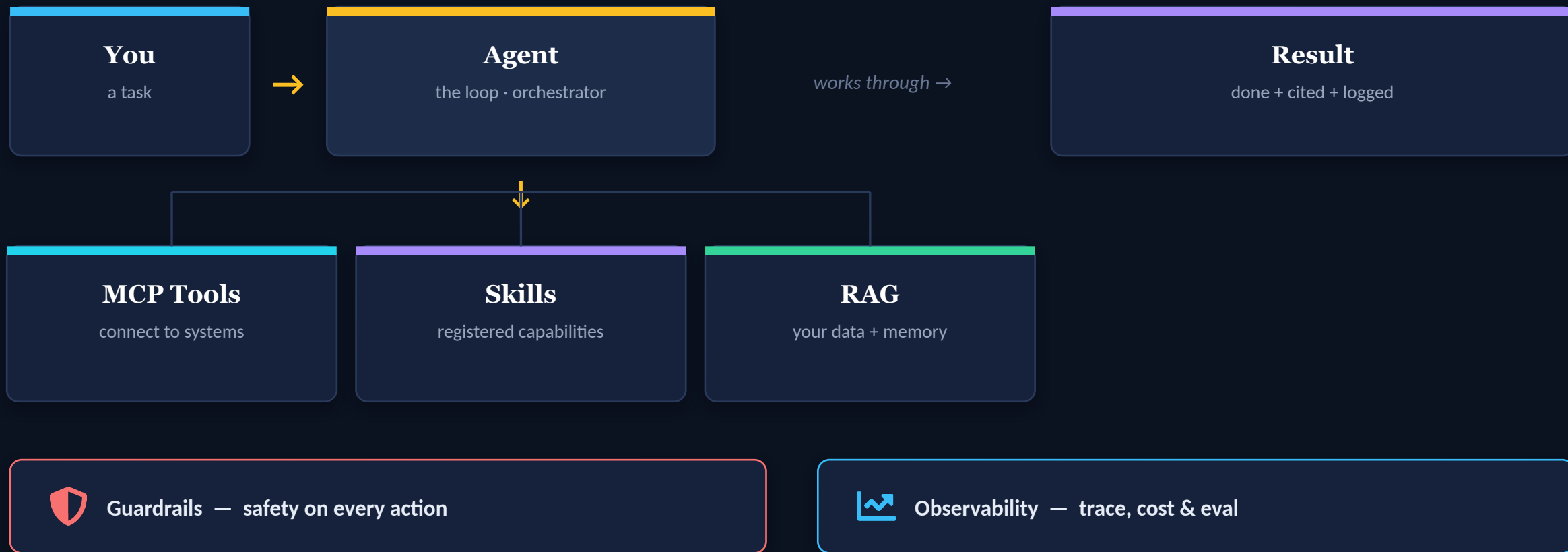


Accuracy levers: chunking · reranking · HyDE · *and measure with RAGAS (faithfulness, relevance) — not vibes.*



PUTTING IT TOGETHER

The assembled AI Workspace



The Agent using its hands (tools)

terminal

```
$ cd Level6_Agent_Patterns  
$ python 04_react_pattern_practical.py
```

WHAT TO WATCH

- Agent decides which tool — we never told it
- Two tools chained for one task (Pillars 1 + 3)
- Production agent prints cost · iterations · tools
- Guards: timeout · budget · stuck-detection

SHOWS PILLARS

1 Agent · 3 Skills/Tools

FILE

Level6_Agent_Patterns/
04_react_pattern_practical.py

NO API KEY?

Runs safe (prints [NO_API_KEY]); walk the trace + show the loop in the editor.

A full workspace — multi-agent code review

PR → AGENTS (parallel) → SYNTHESIZE → MCP ALERT → SHIP

Security Agent

Opus

\$0.08 / PR

Performance Agent

Sonnet

\$0.02 / PR

Style Agent

Haiku

\$0.001 / PR

↓ *synthesize* → critical? human-in-loop → post comments + Slack via MCP

Total ≈ **\$0.10 / PR** — model tiering = ~80× cheaper

ALL 4 PILLARS, LIVE

- Agents — multi-agent supervisor
- MCP — Slack alert tool
- Skills — per-agent review tools
- Routing + human-in-the-loop
- Cost-aware model tiering

`Projects/project3_multiagent_code_review_starter/main.py`

Runs with no key — prints the architecture.



MAKE IT TRUSTWORTHY

Guardrails & observability

A workspace that acts on real systems needs brakes and a dashboard. This is what makes the team — and the manager — trust it.



Guardrails

Block prompt injection, PII leaks, unsafe output.



Observability

Trace every step, token & dollar. LangSmith / Langfuse.



Evaluation

Golden sets + LLM-as-judge, run in CI.



Cost controls

Budgets, caching, model tiering per step.



Human-in-Loop

Approval gates on risky / irreversible actions.



Safe tools

Sandbox, allowlists, read-only, rate limits.



START HERE

Stand up your AI Workspace — in order

- | | | |
|---|----------------------------|------------------------------|
| 1 | Environment | venv + stack + .env keys |
| 2 | First agent loop | ReAct: think → act → observe |
| 3 | Add a tool (skill) | function + schema + registry |
| 4 | Wrap it as MCP | reuse across apps & models |
| 5 | Add RAG | ground it in your data |
| 6 | Guardrails + observability | trust + traces + budgets |
| 7 | Go multi-agent | specialists + supervisor |

THE SHORTCUT

- Pick **ONE** small, low-risk use-case (e.g. internal-docs Q&A).
- Build it with an eval from day one.
- Steps 1–5 get you a working workspace.
- 6–7 make it production-grade.
- Most value lands by step 5 — don't over-build.



RIGHT-SIZE IT

Don't over-engineer the workspace

Add a pillar only when the task needs it. The simplest workspace that solves the problem wins.

If you need: **Just answer from a prompt**

→ **Single LLM call**

If you need: **Answer from our docs/data**

→ **+ RAG**

If you need: **Act on a system (DB, API, Git)**

→ **+ Skills / Tools**

If you need: **Reuse tools across apps/models**

→ **+ MCP**

If you need: **Multi-step, dynamic decisions**

→ **+ Agent loop**

If you need: **Distinct specialties in parallel**

→ **+ Multi-agent**



WRAP-UP

An AI Workspace, in one breath

THE FOUR PILLARS



Agent

reasons, decides, orchestrates the loop.



MCP

standard plug for tools & data.



Skills / Tools

capabilities the agent can invoke.



RAG

grounded in your own data.

TAKE THIS HOME

- A workspace = Agent + MCP + Skills + RAG, working together.
- Build the loop by hand once; you'll understand the whole thing.
- MCP makes a tool reusable everywhere — write once.
- RAG grounds it in YOUR data; guardrails make it safe.
- Start with one low-risk use-case. Most value lands by step 5.

Thank you · Questions?